

ATLAS

Personal Life Operating System

System Design Document -- v0.1.0

Repository: <https://github.com/Swayamjain1240/Atlas>

Table of Contents

- 1. System Overview
- 2. Component Design
- 3. Database Schema Design
- 4. API Design
- 5. Data Flow Design
- 6. Security Design
- 7. UI/UX Design
- 8. Error Handling Strategy
- 9. Performance Design
- 10. Testing Strategy

1. System Overview

Atlas is designed as a local-first monolithic application with four primary subsystems: Ingestion, Storage, AI Engine, and Presentation. The system emphasizes privacy, performance, and offline capability above all else.

Design Principles

- Privacy by Design: All data processing happens locally. Zero telemetry by default.
- Offline-First: Full functionality without internet connection. LLM runs locally.
- Composability: Plugin architecture for custom ingestors, agents, and data sources.
- Performance: Sub-second query response for common queries. Async ingestion pipeline.
- Developer Experience: Clean API, comprehensive documentation, hot-reload development.

2. Component Design

2.1 AtlasAIEngine

The AI Engine is the central component managing RAG operations:

- Embedding: Sentence-Transformers (all-MiniLM-L6-v2) with 384-dim output vectors
- Vector Store: ChromaDB with HNSW indexing for approximate nearest neighbor search
- Retrieval: Cosine similarity search with metadata filtering and hybrid search (planned)
- Generation: Ollama API for local LLM inference (supports Llama 3.2, Mistral, Gemma)
- Context Assembly: Dynamic prompt construction with ranked context snippets
- Memory: Conversation history stored in SQLite for multi-turn coherence

2.2 Ingestion Pipeline

The ingestion pipeline follows a producer-consumer pattern:

- Discoverer: Scans configured sources for new/changed data
- Reader: Reads raw content using source-specific parsers (PyPDF2, python-docx, etc.)
- Chunker: Splits content into semantic chunks (~1500 chars with overlap)
- Embedder: Converts chunks to vector embeddings
- Indexer: Stores embeddings in ChromaDB with metadata
- Scheduler: Optional periodic re-scanning via watchfiles or cron

2.3 API Layer

REST API designed around three core operations: query, ingest, and manage.

- Framework: FastAPI with automatic OpenAPI documentation (/docs)
- Validation: Pydantic models for request/response validation
- Async: All endpoints are async with background task support
- Error Format: Consistent JSON error responses with codes
- Rate Limiting: Token bucket algorithm (100 req/min per client)

3. Database Schema Design

3.1 Vector Store (ChromaDB)

```
Collection: atlas_memory
- id: str (SHA256 hash of source + path, 24 chars)
- embedding: float[384] (Sentence-Transformer vector)
- document: str (chunk text content)
- metadata: dict {
    "source": str ("file" | "browser" | "message" | "email"),
    "source_path": str (original path/URL),
    "timestamp": str (ISO 8601 datetime),
    "tags": list[str] (optional),
}
```

3.2 Metadata Store (SQLite)

```
TABLE memories (
    id TEXT PRIMARY KEY,
    source TEXT NOT NULL,
    source_path TEXT NOT NULL,
    title TEXT,
    text_preview TEXT,
    metadata_json TEXT,
    ingested_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    accessed_count INTEGER DEFAULT 0,
    last_accessed TIMESTAMP
);

TABLE conversations (
    id TEXT PRIMARY KEY,
    session_id TEXT,
    role TEXT, -- "user" | "assistant"
    content TEXT,
    sources_json TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

TABLE config (
    key TEXT PRIMARY KEY,
    value TEXT,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

4. API Design

4.1 Endpoints

GET	/api/v1/health	-> Health check
POST	/api/v1/query	-> Ask a question
POST	/api/v1/ingest	-> Ingest data
GET	/api/v1/stats	-> Memory statistics
GET	/api/v1/memory/recent	-> Recent memories
DELETE	/api/v1/memory/{id}	-> Delete a memory
POST	/api/v1/search	-> Direct vector search

4.2 Standard Response Format

```
{
  "success": true|false,
  "data": { ... },           // present on success
  "error": {                 // present on error
    "code": "ERROR_CODE",
    "message": "Human-readable description"
  },
  "meta": {
    "processing_time_ms": 123.45,
    "version": "0.1.0"
  }
}
```

5. Data Flow Design

5.1 Query Flow

The query flow is the most performance-critical path in Atlas. The system uses a three-stage pipeline: embed -> retrieve -> generate. Each stage is independently cacheable and measurable.

5.2 Ingestion Flow

Ingestion runs as a background process. The pipeline discovers sources, reads content, chunks documents, generates embeddings, and stores results. Each stage is designed to be parallelizable. Failed documents are logged and skipped without blocking the pipeline.

6. Security Design

- Authentication: Local API key stored in environment variable `ATLAS_API_KEY`
- Validation: All inputs sanitized via Pydantic + HTML escaping
- CORS: Restricted to localhost origins only
- Headers: `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Content-Security-Policy: strict`
- Rate Limiting: 100 requests/minute per IP, burst allowance of 20
- Dependencies: Regular npm/pip audit, vulnerability scanning pre-commit
- Secrets: `.env` file in `.gitignore`, pre-commit hook for secret scanning
- Encryption: All data encrypted at rest via OS-level encryption (planned: application-level)

7. UI/UX Design

7.1 Screen Layout (One-Viewport Rule)

Every authenticated screen fits in a single viewport without page-level scrolling. Multiple sections are handled via tabs, pagination, or routing -- never overflow:hidden.

7.2 3D Knowledge Graph

The 3D knowledge graph uses Three.js to render memory nodes as orbiting particles connected by glow lines. Nodes scale based on relevance, recency, and access frequency. Users can click nodes to inspect memories, drag to rotate, and zoom to explore.

7.3 Color Palette

Background:	#0a0a0f (deep space blue-black)
Surface:	#12121a (elevated panels)
Primary:	#6366f1 (atlas indigo)
Text:	#f1f5f9 (light gray)
Accent:	#a78bfa (purple glow)

8. Error Handling Strategy

- All API errors return consistent JSON format with error codes
- Common codes: `VALIDATION_ERROR`, `RATE_LIMITED`, `NOT_FOUND`, `INTERNAL_ERROR`, `UNAUTHORIZED`
- LLM failures fall back to context-only responses without generation
- Ingestion failures are logged and do not block the pipeline
- Frontend shows user-friendly error messages with retry options
- Global exception handler catches unhandled errors

9. Performance Design

- Target: Query response < 2s for 95th percentile
- Embedding: ~50ms per 100 documents (384-dim, CPU)
- Vector Search: < 100ms for 100K vectors (HNSW indexing)
- LLM Generation: ~1-3s for responses (Llama 3.2 3B on decent CPU)
- Ingestion: ~100 documents/second (text files)
- Caching: LRU cache for frequent queries (TTL: 5 minutes)

10. Testing Strategy

- Unit Tests: Pytest for backend (target: 80%+ coverage)
- Integration Tests: API endpoint testing with httpx
- E2E Tests: Playwright for frontend flows
- Security Tests: Bandit for Python, npm audit for JS
- Performance Tests: Locust for load testing API
- Git Hooks: Pre-commit hooks for linting, secrets scan, basic tests